

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут ім. Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЛАБОРАТОРНА РОБОТА № 5

з дисципліни «Системне програмне забезпечення»
на тему «**Файлова система (частина 2)**»

ВИКОНАВ:
Білий І. В.
студент III курсу ФІОТ
групи ІО-34
Залікова № 3402

ПЕРЕВІРИВ:
Андрій Валерійович Сімоненко

ЗАВДАННЯ

Додати до драйвера ФС, який був розроблений у лабораторній роботі №4, підтримку символічних посилань та дерева директорій. Додати в програму команди для роботи з символічними посиланнями та директоріями.

Програма повинна підтримувати шляхові імена у всіх командах (також тих, які були розроблені в лабораторній роботі №4). Якщо остання компонента шляхового імені є символічним посиланням, тоді команди `link` та `unlink` мають працювати з символічним посиланням, тобто на символічне посилання можуть вказувати кілька жорстких посилань. Команда `link` не повинна створювати жорсткі посилання на директорії. Команда `unlink` не повинна працювати з жорсткими посиланнями на директорії.

Додати до програми підтримку наступних команд (замість введення команд можна викликати необхідні функції безпосередньо в програмі):

- `mkdir pathname` – створити директорію та створити відповідне жорстке посилання на неї.
- `rmdir pathname` – звільнити порожню директорію на яке вказує шляхове ім'я та знищити відповідне жорстке посилання на неї (вміст директорії не повинен мати жодного жорсткого посилання, крім наперед визначених жорстких посилань з іменами `.` та `..`).
- `cd pathname` – змінити поточну робочу директорію.
- `symlink str pathname` – створити символічне посилання з вмістом `str` та створити на нього відповідне жорстке посилання. Максимальна довжина вмісту символічного посилання `str` не має перевищувати розмір одного блоку.

У цих командах `pathname` – це шляхове ім'я. У командах з лабораторної роботи №4 замість `name` можна записати `pathname`, для позначення того, що всі команди мають підтримувати роботу з шляховими іменами.

Все, що написано в лабораторній роботі №4, також справедливо для цієї лабораторної роботи

Описи форматів даних ФС

У цій роботі я суттєво розширив можливості своєї ФС, перейшовши від однієї плоскості директорії до повноцінного дерева та додавши нові типи об'єктів. Я замінив фіксований масив на `std::unordered_map`. Тепер замість того, щоб резервувати 10 000 блоків одразу, пам'ять під блок (128 байт) виділяється тільки в момент реального запису у нього. Це дозволило мені відмовитися від бітової карти блоків, бо тепер наявність ключа в мапі і означає, що блок зайнятий. Оновлений дескриптор файлу `Inode`, я додав підтримку трьох типів файлів: `TYPE_REG (0)` - звичайний файл, `TYPE_DIR (1)` - директорія, та `TYPE_SYM (2)` - символічне посилання. Для директорій `size` не використовується для даних, а натомість структура зберігається у спеціальній таблиці директорій. Для символічних посилань вміст рядок шляху зберігається безпосередньо в першому блоці даних (`direct[0]`). Довжина такого рядка жорстко обмежена 128 байтами, щоб він гарантовано займав рівно один блок і не викликав переповнення пам'яті.

Ієрархічне дерево директорій (directories): Тепер це не один список, а велика таблиця `std::unordered_map<int, std::map<std::string, int>>`. Вона працює так: по id директорії ми отримуємо словник, де ключі це імена файлів, а значення id їхніх дескрипторів. При створенні будь-якої директорії я автоматично додаю два жорстких посилання: `.` (посилання на себе) та `..` (посилання на «батьківську» директорію).

Механізм пошуку шляхів `resolve`: Це логічний центр системи. Я реалізував алгоритм, який розбиває шлях на частини та послідовно "спускається" по дереву. Якщо на шляху зустрічається символічне посилання, парсер автоматично зчитує його вміст і продовжує пошук вже за новою адресою.

Також у командах `rmdir` та `unlink` я додав перевірку імені компонента. Це гарантує, що користувач не зможе видалити системні записи `.` або `..`, що інакше призвело б до розриву зв'язків у дереві та втрати доступу до частин ФС.

Лістинг програми:

```
/*
 * Лабораторна робота №5
 * Дисципліна: Системне програмне забезпечення
 * Тема: Файлова система (частина 2)
 * Студент: Білий Іван Віталійович, група ІО-34
 */

#include <iostream>
#include <vector>
#include <string>
#include <map>
#include <unordered_map>
#include <array>
#include <cstring>
#include <iomanip>
#include <algorithm>
#include <sstream>
#include <list>

const int BLOCK_SIZE = 128;
const int DIRECT_PTRS = 4; // кількість прямих посилань
const int PTRS_PER_BLOCK = BLOCK_SIZE / sizeof(int); // скільки посилань влізе в один блок

const int TYPE_REG = 0; // звичайний файл
const int TYPE_DIR = 1; // директорія
const int TYPE_SYM = 2; // новий тип для символічних посилань

// фіксований розмір блоку
using Block = std::array<char, BLOCK_SIZE>;
```

```

// Дескриптор файлу
struct Inode {
    int id;
    int type;
    int nlink;
    int size;
    int direct[DIRECT_PTRS]; // прямі посилання на блоки
    int indirect;             // непряме посилання
    int open_count;           // скільки разів файл зараз відкрито

    // конструктор за замовчуванням
    Inode() : id(-1), type(TYPE_REG), nlink(0), size(0), indirect(-1),
open_count(0) {
        for (int i = 0; i < DIRECT_PTRS; ++i) direct[i] = -1;
    }

    // рахую скільки фізичних блоків реально виділено під файл
    int allocated_blocks(const std::unordered_map<int, Block>& disk) const {
        int count = 0;
        for (int i = 0; i < DIRECT_PTRS; ++i) {
            if (direct[i] != -1) count++;
        }
        if (indirect != -1) {
            count++; // враховує сам блок з покажчиками
            if (disk.find(indirect) != disk.end()) {
                const int* ind_ptrs = reinterpret_cast<const
int*>(disk.at(indirect).data());
                for (int i = 0; i < PTRS_PER_BLOCK; ++i) {
                    if (ind_ptrs[i] != -1) count++;
                }
            }
        }
        return count;
    }
};

// структура для відкритого файлу
struct OpenFile {
    int inode_id;
    int offset;
    bool is_open;
    OpenFile() : inode_id(-1), offset(0), is_open(false) {}
};

// головний клас нашої ФС
class FileSystem {
private:
    std::unordered_map<int, Block> disk; // оптимізована імітація диску, виділяє
пам'ять тільки коли треба
    int next_free_block_id; // лічильник для видачі унікальних ID блокам

```

```

std::vector<Inode> inodes;           // масив усіх інодів
std::vector<bool> inode_bitmap;     // бітова карта вільних інодів

// дерево директорій id_батька -> ім'я_файлу -> id_дитини
std::unordered_map<int, std::map<std::string, int>>> directories;

std::vector<OpenFile> open_files;   // таблиця відкритих файлів

int root_id; // id кореневої директорії /
int cwd_id;  // id поточної робочої директорії CWD

// просто видає новий номер
int alloc_block() {
    return next_free_block_id++;
}

// звільняє блок
void free_block(int bno) {
    disk.erase(bno);
}

// шукає вільний inode
int alloc_inode() {
    for (size_t i = 0; i < inode_bitmap.size(); ++i) {
        if (!inode_bitmap[i]) {
            inode_bitmap[i] = true;
            inodes[i] = Inode();
            inodes[i].id = i;
            return i;
        }
    }
    return -1;
}

// перетворює логічний блок файлу у фізичний якщо allocate=true то виділяє
// новий
int get_logical_block(Inode& inode, int lbn, bool allocate) {
    if (lbn < DIRECT_PTRS) {
        if (inode.direct[lbn] == -1 && allocate) {
            inode.direct[lbn] = alloc_block();
        }
        return inode.direct[lbn];
    } else {
        // робота з непрямыми посиланнями
        if (inode.indirect == -1) {
            if (!allocate) return -1;
            inode.indirect = alloc_block();
            int* ind_ptrs =
reinterpret_cast<int*>(disk[inode.indirect].data());
            for (int i = 0; i < PTRS_PER_BLOCK; ++i) ind_ptrs[i] = -1;
        }
    }
}

```

```

        int ind_idx = lbn - DIRECT_PTRS;
        if (ind_idx >= PTRS_PER_BLOCK) return -1; // файл занадто великий

        int* ind_ptrs = reinterpret_cast<int*>(disk[inode.indirect].data());
        if (ind_ptrs[ind_idx] == -1 && allocate) {
            ind_ptrs[ind_idx] = alloc_block();
        }
        return ind_ptrs[ind_idx];
    }
}

// очищує всі блоки файлу прямі і непрямі
void free_file_blocks(Inode& inode) {
    for (int i = 0; i < DIRECT_PTRS; ++i) {
        if (inode.direct[i] != -1) {
            free_block(inode.direct[i]);
            inode.direct[i] = -1;
        }
    }
    if (inode.indirect != -1) {
        if (disk.find(inode.indirect) != disk.end()) {
            int* ind_ptrs =
reinterpret_cast<int*>(disk[inode.indirect].data());
            for (int i = 0; i < PTRS_PER_BLOCK; ++i) {
                if (ind_ptrs[i] != -1) free_block(ind_ptrs[i]);
            }
        }
        free_block(inode.indirect);
        inode.indirect = -1;
    }
}

// якщо на файл немає посилань і він закритий то видаляє назавжди
void check_free_inode(int inode_id) {
    if (inodes[inode_id].nlink <= 0 && inodes[inode_id].open_count == 0) {
        free_file_blocks(inodes[inode_id]);
        directories.erase(inode_id); // чистить вміст, якщо це була
директорія
        inode_bitmap[inode_id] = false;
    }
}

// парсер шляхів
int resolve(const std::string& path, int& parent_id, std::string& basename,
bool follow_last) {
    // якщо шлях починається з /, йде від кореня інакше від поточної
директорії
    int curr = (path.empty() || path[0] != '/') ? cwd_id : root_id;
    parent_id = curr;
    basename = "";
}

```

```

// розбиваю шлях на компоненти по слешу
std::list<std::string> parts;
std::stringstream ss(path);
std::string item;
while (std::getline(ss, item, '/')) {
    if (!item.empty() && item != ".") parts.push_back(item);
}

int symlinks_followed = 0;

while (!parts.empty()) {
    std::string part = parts.front();
    parts.pop_front();

    parent_id = curr;
    basename = part;

    if (inodes[curr].type != TYPE_DIR) return -1;

    auto& dir = directories[curr];
    if (dir.find(part) == dir.end()) {
        if (parts.empty()) return -1;
        parent_id = -1;
        return -1;
    }

    int next_id = dir[part];
    bool is_last = parts.empty();

    // якщо натрапив на симлінк і має за ним йти
    if (inodes[next_id].type == TYPE_SYM && (!is_last || follow_last)) {
        if (++symlinks_followed > 10) { // захист від нескінченних циклів
            std::cout << "Помилка: забагато рівнів символічних
посилань\n";
            return -1;
        }

        std::string sym_path = "";
        int bno = inodes[next_id].direct[0];
        if (bno != -1 && disk.find(bno) != disk.end()) {
            sym_path = std::string(disk[bno].data(),
inodes[next_id].size);
        }

        // якщо симлінк абсолютний то стрибає в корінь
        if (!sym_path.empty() && sym_path[0] == '/') {
            curr = root_id;
        }

        // розбиваю шлях з симлінку і додаю його на початок черги
        std::vector<std::string> sym_parts;

```

```

        std::stringstream ss_sym(sym_path);
        while (std::getline(ss_sym, item, '/')) {
            if (!item.empty() && item != ".") sym_parts.push_back(item);
        }

        for (auto it = sym_parts.rbegin(); it != sym_parts.rend(); ++it)
    {
        parts.push_front(*it);
    }
    } else {
        curr = next_id;
    }
}

if (basename.empty()) {
    parent_id = curr;
    basename = ".";
}
return curr;
}

public:
    // ініціалізація ФС
    void mkfs(int n) {
        inodes.assign(n, Inode());
        inode_bitmap.assign(n, false);
        disk.clear();
        directories.clear();
        next_free_block_id = 0;
        open_files.assign(256, OpenFile());

        // створює кореневу директорію /
        root_id = alloc_inode();
        inodes[root_id].type = TYPE_DIR;
        inodes[root_id].nlink = 2; // сама на себе (.) та (..)
        directories[root_id]["."] = root_id;
        directories[root_id][".."] = root_id;
        cwd_id = root_id;

        std::cout << "ФС ініціалізована з " << n << " інодами.\n";
    }

    // створює нову директорію
    void mkdir(const std::string& path) {
        int parent, target; std::string base;
        target = resolve(path, parent, base, false);
        if (target != -1) { std::cout << "Помилка: цільове ім'я вже існує\n";
return; }
        if (parent == -1 || base.empty() || base == "." || base == "..") {
            std::cout << "Помилка: невірний шлях\n"; return;
        }
    }

```



```

    int id = alloc_inode();
    if (id == -1) { std::cout << "Помилка: немає вільних інодів\n"; return; }

    inodes[id].type = TYPE_DIR;
    inodes[id].nlink = 2;
    directories[id]["."] = id;
    directories[id][".."] = parent;
    directories[parent][base] = id;
    inodes[parent].nlink++; // .. нового нащадка вказує на parent
}

// видаляє порожню директорію
void rmdir(const std::string& path) {
    int parent, target; std::string base;
    target = resolve(path, parent, base, false);

    if (target == -1) { std::cout << "Помилка: файл не знайдено\n"; return; }
    if (target == root_id) { std::cout << "Помилка: не можна видалити
корінь\n"; return; }

    // забороняє видаляти системні посилання . та ..
    if (base == "." || base == "..") {
        std::cout << "Помилка: не можна видаляти системні посилання . та
..\n";
        return;
    }

    if (inodes[target].type != TYPE_DIR) { std::cout << "Помилка: це не
директорія\n"; return; }
    if (directories[target].size() > 2) { std::cout << "Помилка: директорія
не порожня\n"; return; }

    directories[parent].erase(base);
    inodes[parent].nlink--; // видаляє посилання ".."
    inodes[target].nlink -= 2; // видаляє "." та зв'язок з батьком
    check_free_inode(target);
}

// змінює поточну робочу директорію
void cd(const std::string& path) {
    int parent, target; std::string base;
    target = resolve(path, parent, base, true);
    if (target == -1) { std::cout << "Помилка: файл не знайдено\n"; return; }
    if (inodes[target].type != TYPE_DIR) { std::cout << "Помилка: це не
директорія\n"; return; }
    cwd_id = target;
}

// створює символічне посилання
void symlink(const std::string& str, const std::string& path) {

```

```

        // Захист від переповнення буфера
        if (str.length() > BLOCK_SIZE) {
            std::cout << "Помилка: довжина символічного посилання перевищує
розмір блоку (" << BLOCK_SIZE << " байт)\n";
            return;
        }

        int parent, target; std::string base;
        target = resolve(path, parent, base, false);
        if (target != -1) { std::cout << "Помилка: цільове ім'я вже існує\n";
return; }
        if (parent == -1 || base.empty()) { std::cout << "Помилка: невірний
шлях\n"; return; }

        int id = alloc_inode();
        if (id == -1) { std::cout << "Помилка: немає вільних інодів\n"; return; }

        inodes[id].type = TYPE_SYM;
        inodes[id].nlink = 1;
        directories[parent][base] = id;

        // зберігає шлях у першому блоці файлу
        inodes[id].size = str.length();
        int bno = alloc_block();
        inodes[id].direct[0] = bno;
        std::copy(str.begin(), str.end(), disk[bno].begin());
    }

    void stat(const std::string& path) {
        int parent, target; std::string base;
        // stat працює з symlink як lstat
        target = resolve(path, parent, base, false);
        if (target == -1) { std::cout << "Помилка: файл не знайдено\n"; return; }

        const Inode& in = inodes[target];
        std::string t_str = (in.type == TYPE_REG) ? "reg" : (in.type == TYPE_DIR
? "dir" : "sym");
        std::cout << "id=" << target << ", type=" << t_str
            << ", nlink=" << in.nlink << ", size=" << in.size
            << ", nblock=" << in.allocated_blocks(disk) << "\n";
    }

    void ls(std::string path = "") {
        if (path.empty()) path = ".";
        int parent, target; std::string base;
        target = resolve(path, parent, base, true);
        if (target == -1) { std::cout << "Помилка: файл не знайдено\n"; return; }
        if (inodes[target].type != TYPE_DIR) { std::cout << "Помилка: це не
директорія\n"; return; }

        for (const auto& pair : directories[target]) {

```

```

        int id = pair.second;
        std::string t_str = (inodes[id].type == TYPE_REG) ? "reg" :
(inodes[id].type == TYPE_DIR ? "dir" : "sym");
        std::cout << pair.first << "\t=> " << t_str << ", " << id;

        // якщо це symlink показує куди він вказує
        if (inodes[id].type == TYPE_SYM) {
            int bno = inodes[id].direct[0];
            if (bno != -1 && disk.find(bno) != disk.end()) {
                std::cout << " -> " << std::string(disk[bno].data(),
inodes[id].size);
            }
        }
        std::cout << "\n";
    }
}

void create(const std::string& path) {
    int parent, target; std::string base;
    target = resolve(path, parent, base, false);
    if (target != -1) { std::cout << "Помилка: файл вже існує\n"; return; }
    if (parent == -1 || base.empty()) { std::cout << "Помилка: невірний
шлях\n"; return; }

    int id = alloc_inode();
    if (id == -1) { std::cout << "Помилка: немає вільних інодів\n"; return; }

    inodes[id].type = TYPE_REG;
    inodes[id].nlink = 1;
    directories[parent][base] = id;
}

void open(const std::string& path) {
    int parent, target; std::string base;
    target = resolve(path, parent, base, true); // відкриває кінцевий файл а
не symlink
    if (target == -1) { std::cout << "Помилка: файл не знайдено\n"; return; }
    if (inodes[target].type == TYPE_DIR) { std::cout << "Помилка: не можна
відкрити директорію\n"; return; }

    int fd = -1;
    // шукає найменший вільний дескриптор
    for (size_t i = 0; i < open_files.size(); ++i) {
        if (!open_files[i].is_open) { fd = i; break; }
    }
    if (fd == -1) { std::cout << "Помилка: забагато відкритих файлів\n";
return; }

    open_files[fd].inode_id = target;
    open_files[fd].offset = 0;
    open_files[fd].is_open = true;

```

```

        inodes[target].open_count++;
        std::cout << "fd = " << fd << "\n";
    }

    void close(int fd) {
        if (fd < 0 || fd >= open_files.size() || !open_files[fd].is_open) {
            std::cout << "Помилка: невірний fd\n"; return;
        }
        int id = open_files[fd].inode_id;
        open_files[fd].is_open = false;
        inodes[id].open_count--;
        check_free_inode(id);
    }

    void seek(int fd, int offset) {
        if (fd < 0 || fd >= open_files.size() || !open_files[fd].is_open) {
            std::cout << "Помилка: невірний fd\n"; return;
        }
        open_files[fd].offset = offset;
    }

    void read(int fd, int size) {
        if (fd < 0 || fd >= open_files.size() || !open_files[fd].is_open) {
            std::cout << "Помилка: невірний fd\n"; return;
        }
        int id = open_files[fd].inode_id;
        Inode& in = inodes[id];

        int read_bytes = 0;
        std::string result = "";

        while (read_bytes < size && open_files[fd].offset < in.size) {
            int lbn = open_files[fd].offset / BLOCK_SIZE;
            int blk_off = open_files[fd].offset % BLOCK_SIZE;
            int pbn = get_logical_block(in, lbn, false);

            int to_read = std::min(size - read_bytes, BLOCK_SIZE - blk_off);
            to_read = std::min(to_read, in.size - open_files[fd].offset);

            // якщо фізичного блоку немає читає нулі
            if (pbn == -1 || disk.find(pbn) == disk.end()) {
                for (int i = 0; i < to_read; ++i) result += "\\0";
            } else {
                for (int i = 0; i < to_read; ++i) {
                    char c = disk[pbn][blk_off + i];
                    if (c == 0) result += "\\0";
                    else result += c;
                }
            }
            read_bytes += to_read;
            open_files[fd].offset += to_read;
        }
    }

```

```

    }
    std::cout << result << "\n";
}

void write(int fd, int size, const std::string& data) {
    if (fd < 0 || fd >= open_files.size() || !open_files[fd].is_open) {
        std::cout << "Помилка: невірний fd\n"; return;
    }
    int id = open_files[fd].inode_id;
    Inode& in = inodes[id];

    int written = 0;
    int data_idx = 0;

    while (written < size) {
        int lbn = open_files[fd].offset / BLOCK_SIZE;
        int blk_off = open_files[fd].offset % BLOCK_SIZE;
        int pbn = get_logical_block(in, lbn, true); // тут allocate=true

        if (pbn == -1) {
            std::cout << "Помилка: файл занадто великий\n"; break;
        }

        int to_write = std::min(size - written, BLOCK_SIZE - blk_off);
        for (int i = 0; i < to_write; ++i) {
            disk[pbn][blk_off + i] = (data_idx < data.length()) ?
data[data_idx++] : 0;
        }

        written += to_write;
        open_files[fd].offset += to_write;
        if (open_files[fd].offset > in.size) {
            in.size = open_files[fd].offset; // оновлює розмір файлу
        }
    }
}

void link(const std::string& path1, const std::string& path2) {
    int p1, target1, p2, target2; std::string b1, b2;
    target1 = resolve(path1, p1, b1, false); // link працює з symlink як з
файлом
    if (target1 == -1) { std::cout << "Помилка: вихідний файл не знайдено\n";
return; }
    if (inodes[target1].type == TYPE_DIR) { std::cout << "Помилка: не можна
робити link на директорію\n"; return; }

    target2 = resolve(path2, p2, b2, false);
    if (target2 != -1) { std::cout << "Помилка: цільове ім'я вже існує\n";
return; }
    if (p2 == -1 || b2.empty()) { std::cout << "Помилка: невірний шлях\n";
return; }
}

```

```

        directories[p2][b2] = target1;
        inodes[target1].nlink++;
    }

    void unlink(const std::string& path) {
        int parent, target; std::string base;
        target = resolve(path, parent, base, false); // unlink не слідує за
symlink
        if (target == -1) { std::cout << "Помилка: файл не знайдено\n"; return; }

        // захист для unlink
        if (base == "." || base == "..") {
            std::cout << "Помилка: не можна видаляти системні посилання . та
..\n";
            return;
        }

        if (inodes[target].type == TYPE_DIR) { std::cout << "Помилка: не можна
робити unlink директорії (використовуйте rmdir)\n"; return; }

        directories[parent].erase(base);
        inodes[target].nlink--;
        check_free_inode(target);
    }

    // оптимізована зміна розміру
    void truncate(const std::string& path, int size) {
        int parent, target; std::string base;
        target = resolve(path, parent, base, true);
        if (target == -1) { std::cout << "Помилка: файл не знайдено\n"; return; }
        if (inodes[target].type == TYPE_DIR) { std::cout << "Помилка: не можна
робити truncate директорії\n"; return; }

        Inode& in = inodes[target];
        // якщо розмір зменшується то чисте зайві фізичні блоки
        if (size < in.size) {
            int new_lbn_end = (size + BLOCK_SIZE - 1) / BLOCK_SIZE;
            for (int i = new_lbn_end; i < DIRECT_PTRS; ++i) {
                if (in.direct[i] != -1) { free_block(in.direct[i]); in.direct[i]
= -1; }
            }
            if (in.indirect != -1 && disk.find(in.indirect) != disk.end()) {
                int* ind_ptrs = reinterpret_cast<int*>(disk[in.indirect].data());
                bool keep_indirect = false;
                for (int i = 0; i < PTRS_PER_BLOCK; ++i) {
                    if (DIRECT_PTRS + i >= new_lbn_end) {
                        if (ind_ptrs[i] != -1) { free_block(ind_ptrs[i]);
ind_ptrs[i] = -1; }
                    } else if (ind_ptrs[i] != -1) {
                        keep_indirect = true;

```

```

        }
    }
    if (!keep_indirect) { free_block(in.indirect); in.indirect = -1;
}

    }

}
// якщо збільшується то просто міняє in.size блоки не виділяє
in.size = size;
}
};

int main() {
    FileSystem fs;
    std::string line;

    std::cout << "Інтерактивна оболонка ФС (Лаб 5)\n> ";

    while (std::getline(std::cin, line)) {
        if (line.empty()) { std::cout << "> "; continue; }

        std::stringstream ss(line);
        std::string cmd;
        ss >> cmd;

        if (cmd == "mkfs") {
            int n; ss >> n; fs.mkfs(n);
        } else if (cmd == "mkdir") {
            std::string path; ss >> path; fs.mkdir(path);
        } else if (cmd == "rmdir") {
            std::string path; ss >> path; fs.rmdir(path);
        } else if (cmd == "cd") {
            std::string path; ss >> path; fs.cd(path);
        } else if (cmd == "symlink") {
            std::string str, path; ss >> str >> path; fs.symlink(str, path);
        } else if (cmd == "create") {
            std::string path; ss >> path; fs.create(path);
        } else if (cmd == "stat") {
            std::string path; ss >> path; fs.stat(path);
        } else if (cmd == "ls") {
            std::string path = "";
            if (ss >> path) fs.ls(path);
            else fs.ls();
        } else if (cmd == "link") {
            std::string p1, p2; ss >> p1 >> p2; fs.link(p1, p2);
        } else if (cmd == "unlink") {
            std::string path; ss >> path; fs.unlink(path);
        } else if (cmd == "open") {
            std::string path; ss >> path; fs.open(path);
        } else if (cmd == "close") {
            int fd; ss >> fd; fs.close(fd);
        } else if (cmd == "seek") {

```

```

        int fd, off; ss >> fd >> off; fs.seek(fd, off);
    } else if (cmd == "read") {
        int fd, size; ss >> fd >> size; fs.read(fd, size);
    } else if (cmd == "write") {
        int fd, size; ss >> fd >> size;
        char sep; ss.get(sep); // забирає пробіл після size
        std::string data; data.resize(size);
        ss.read(&data[0], size);
        fs.write(fd, size, data);
    } else if (cmd == "truncate") {
        std::string path; int size; ss >> path >> size; fs.truncate(path,
size);
    } else if (cmd == "exit") {
        break;
    } else {
        std::cout << "Невідома команда\n";
    }
    std::cout << "> ";
}
return 0;
}

```

Результати тестування програми:


```
bilyi-ivan-io-34@bilyi-ivan-io-34-VirtualBox: ~/lab5_fs
bilyi-ivan-io-34@bilyi-ivan-io-34-VirtualBox:~/lab5_fs$ ./fs_lab5
Інтерактивна оболонка ФС (Лаб 5)
> mkfs 100
ФС ініціалізована з 100 інодами.
> mkdir dir1
> mkdir dir1/dir2
> stat dir1
id=1, type=dir, nlink=3, size=0, nblock=0
> ls
.      => dir, 0
..     => dir, 0
dir1   => dir, 1
> ls
.      => dir, 0
..     => dir, 0
dir1   => dir, 1
> cd dir1/dir2
> create file.txt
> mkdir /a
> mkdir /a/b
> symlink /dir1 /a/b/l1
> ls /a/b
.      => dir, 5
..     => dir, 4
l1     => sym, 6 -> /dir1
> open /a/b/l1/dir2/file.txt
fd = 0
> symlink dir2/../../dir1/./dir2/file.txt /dir1/l2
> cd /a/b
> open l1/l2
fd = 1
> cd /
> unlink /dir1/l2
> unlink /a/b/l1
> unlink dir1/dir2/file.txt
> unlink /dir1/dir2
Помилка: не можна робити unlink директорії (використовуйте rmdir)
```

Висновок:

Під час виконання лабораторної роботи №5 я розширив можливості розробленої раніше файлової системи, реалізувавши підтримку ієрархічної структури директорій та символічних посилань.

У результаті роботи було досягнуто наступних цілей:

- Реалізовано деревоподібну структуру ФС: тепер система підтримує необмежену вкладеність директорій, де кожна папка коректно обробляє системні жорсткі посилання . (поточна директорія) та .. (батьківська директорія).
- Впроваджено підтримку шляхових імен: розроблений механізм resolve дозволяє працювати як з абсолютними шляхами від кореня /, так і з відносними шляхами від поточної робочої директорії CWD.

- Додано символічні посилання: реалізовано новий тип файлу, що зберігає шлях до іншого об'єкта. Я імплементував алгоритм автоматичного розкриття симлінків під час парсингу шляху з механізмом захисту від нескінченних циклів.
- Забезпечено цілісність даних: у командах `rmdir` та `unlink` було додано критично важливі перевірки, які забороняють видалення системних посилань `.` та `..`, а також захищають директорії від випадкового видалення через `unlink`.
- Оптимізовано використання ресурсів: перехід на `std::unordered_map` для імітації дискового простору дозволив реалізувати розріджені файли на рівні блоків пам'яті. Тепер пам'ять виділяється динамічно лише для тих блоків, куди реально записуються дані.
- Виконано вимоги безпеки: додано перевірку довжини вмісту символічного посилання (не більше 128 байт), що запобігає переповненню буфера та відповідає технічному завданню.

Посилання на github: https://github.com/IvanBilyi/SS_Lab5